

# Deep Dive into Python OOP: How Inheritance Affects the Design of Model Compression

## 1. Introduction

요즘 연구들을 보면, overparametered 된 모델들이 많다. 예를 들면 LLM 같은 모델들은 요즘 100B도 넘는 모델들도 나오기 때문에, model compression을 통해서 edge device에서의 pre-training/fine-tuning/inference를 가능할 수 있도록 하는 것이 각광받고 있다. Model compression method에는 대표적으로 영향력이 낮은 weight를 잘라내는 pruning, weight나 activation을 32FP에서 8-bit, 4-bit 등으로 저장하는 quantization, 파라미터가 많은 teacher 모델의 분포를 파라미터가 적은 student 모델이 배우도록 하는 Knowledge Distillation(KD) 방식이 있다.

위와 같은 model compression을 하다 보면 다양한 종류의 레이어와 복잡한 경량화 기법을 하나의 프로그램에 구현하게 된다. 그러다 보면 코드가 얹히고설켜 유지보수가 불가능해지는 스파게티 코드가 발생하게 된다.

본 보고서에서는 이를 해결하기 위한 Python OOP의 핵심 개념인 상속이 이러한 문제를 해결하고, 확장이 가능하며 모듈화된 경량화 프레임워크를 구축하는 가장 혁신적인 해결책임을 증명하고자 한다.

## 2. Inheritance

새로운 클래스(자식)가 기존 클래스(부모)의 속성과 메서드를 그대로 물려받아 코드의 재사용성을 극대화하는 메커니즘이다.

부모에게 물려받은 메서드를 자식 클래스가 자신의 목적에 맞게 overriding할 수 있으며, 이를 통해 동일한 명령으로 서로 다른 동작을 수행하게 하는 polymorphism을 자연스럽게 확보할 수 있다.

## 3. Implementation

실험은 간단한 layer를 통하는 forward 함수에 대해서 구현해보았다.

실험 1은 상속 없이 기본/pruning/quantization을 구현하였고, 실험 2는 똑같은 실험을 상속과 다형성을 활용하여 구현하였다.

### A. Experiment 1(절차지향)

```
4. def forward(x, weights, compression, threshold=0.0):
5.
6.     if compression == "basic":
7.         return [w * x for w in weights]
8.
9.     elif compression == "pruning":
10.        return [(w if abs(w) > threshold else 0.0) * x for w in
11.                weights]
```

```

12.     elif compression == "quantization":
13.         return [round(w) * x for w in weights]
14.
15.     else:
16.         raise ValueError("Unknown layer type")
17.
18. weights = [0.1, -0.8, 0.4, -0.2]
19.
20. print(f"basic forward : {forward(2.0, weights, "basic",
    threshold=0.3)}")
21. print(f"pruning forward : {forward(2.0, weights, "pruning",
    threshold=0.3)}")
22. print(f"quantization forward : {forward(2.0, weights, "quantization",
    threshold=0.3)}")

```

(실험 코드 1)

```

basic forward : [0.2, -1.6, 0.8, -0.4]
pruning forward : [0.0, -1.6, 0.8, 0.0]
quantization forward : [0.0, -2.0, 0.0, 0.0]

```

(실험 결과 1)

B. Experiment 2(객체지향-상속 활용)

```

C. class BaseLayer:
D.     def __init__(self, name, weights):
E.         self.name = name
F.         self.weights = weights
G.
H.     def forward(self, x):
I.         raise NotImplementedError("Subclasses must implement forward
    method.")
J.
K. class BasicLayer(BaseLayer):
L.     def forward(self, x):
M.         return [w * x for w in self.weights]
N.
O. class PrunedLayer(BaseLayer):
P.     def __init__(self, name, weights, threshold=0.5):
Q.         super().__init__(name, weights)
R.         self.threshold = threshold
S.
T.     def forward(self, x):
U.         return [(w if abs(w) > self.threshold else 0.0) * x for w in
    self.weights]
V.
W. class QuantizedLayer(BaseLayer):
X.     def forward(self, x):
Y.         return [round(w) * x for w in self.weights]
Z.
AA. layers = [

```

```

BB.     BasicLayer("basic forward", [0.1, -0.8, 0.4, -0.2]),
CC.     PrunedLayer("pruning forward", [0.1, -0.8, 0.4, -0.2],
        threshold=0.3),
DD.     QuantizedLayer("quantization forward", [0.1, -0.8, 0.4, -0.2])
EE. ]
FF.
GG. for layer in layers:
HH.     print(f"{layer.name} : {layer.forward(2.0)}")

```

(실험 코드 2)

```

basic forward : [0.2, -1.6, 0.8, -0.4]
pruning forward : [0.0, -1.6, 0.8, 0.0]
quantization forward : [0.0, -2.0, 0.0, 0.0]

```

(실험 결과 2)

### C. Comparative Analysis

실험 1의 결과를 보면, basic/pruning/quantization 각각의 경우에 대해 if와 elif, else로 case를 다 나눠서 코드를 구현해야 하는 것을 볼 수 있다.

그러나, 상속을 활용한 실험 2를 보게 되면, if와 elif는 하나도 없이 부모 클래스의 함수를 오버라이드하였기 때문에 실제 함수를 호출하는 과정에서 layer.forward(2.0)만 호출해도 되는 것을 볼 수 있다.

그리고 만약 이 코드에 KD 기반 압축이라는 기능을 추가로 넣는다고 가정하면, 절차지향적 방식으로는 forward 함수에 elif로 KD case를 구분하도록 구현해야 한다. 그러나 OOP에서는, 기존 코드를 수정할 필요 없이, 새로운 KD 클래스를 구현하면 된다.

### 4. Discussion

코드를 구현하면서 느낀 점인데, 함수가 간단한 단일 연산이기 때문에 코드 길이는 사실 실험 2가 더 길지만, LLM과 같은 큰 모델들을 구현하게 된다면 코드가 복잡해져 상속을 활용한 것이 코드 길이는 물론이고 확장성과 안전성 측면에서 이점이 훨씬 크다는 것을 <> 논문을 통해 확인할 수 있었다.

### 5. Related Work

<> 논문